Camunda 8 - Getting Started with Decision Automation

Business Requirement

We will be using a fictitious organization; Camundanzia, as the scenario for this course.

Camundanzia

Scenario

The Camundanzia insurance company has successfully implemented **Process Orchestration** throughout their organization. As a next step, they would like to introduce **Decision Automation** using Decision Model and Notation (DMN).

Camundanzia would like to enhance their existing *Request Car Insurance* process with automated decisions using DMN. The current version of the process no longer meets the business requirements as it only considers *Driver Experience*.

The updated version of the process will also need to consider *Car Type*. In addition, the **Insurance Manager** also expects to make regular changes to the **Policy Rules** in the future. Therefore, the new version of the *Request Car Insurance* process must be easy to maintain.

User Story

Three simple **user stories** to describe the requirements can be found below:

User Story

- As an **Insurance Manager**
- I want to update **Policy Rules** without the need for a **Developer**
- So that I can ensure that Camundanzia's exposure to risk is minimized

User Story

- As an **Insurance Manager**
- I want to automatically approve requests from experienced drivers with small cars
- So that my **Insurance Specialist** can focus on more complex *Requests*

User Story

- As an **Insurance Manager**
- I want to automatically reject requests from inexperienced drivers with luxury cars
- So that my **Insurance Specialist** can focus on more complex *Requests*

Acceptance Criteria

We will take an MVP approach to developing an application that meets these requirements.

The initial acceptance criteria that we will be working to are:

- Requests should be rejected for a *Family* or *Luxury* car if the **Driver** has less than 2 years of experience
- Requests should be accepted for a *Small* car if the **Driver** has more than 2 years of experience
- Requests should be accepted if the **Driver** has more than 4 years of experience
- All other requests will be determined by an **Insurance Specialist**
- The **Insurance Manager** can review each *Request* after completion

Iterative Development

Iterative Development

Camunda is well suited to an iterative development approach; applications can be easily revised and upgraded without affecting in-flight processes.

Outline Solution

We now have a clear set of requirements and acceptance criteria for the Camundanzia *Request Car Insurance* process.

The sections below will provide you with an overview of how to develop a **Decision Automation** solution that meets these requirements.

Camundanzia

Scenario

Imagine that you have recently joined Camundanzia as a new Developer and have been assigned to the Camunda project team. Your first assignment is to enhance the *Request Car Insurance* process using the requirements from the previous lesson.

Fortunately for you, a more experienced Camunda Developer has drafted an outline solution for you. Your task is to implement a solution based upon the outline that has been provided.

Outline Solution

For each of the acceptance criteria that have been identified, the more experienced Developer has provided a summary of the Camunda components that provide the implementation of the requirements in addition to the configuration that is required by you, the Developer.

Implement Decision Automation

We first need to implement **Decision Automation** using the DMN functionality provided in Camunda. **Business Rules** will be configured in a **Decision Table** contained in a **Decision Requirements Diagram (DRD)**.

- **Decision Requirements Diagrams (DRDs)** are created using the Modeler
- **Decision Tables** are created using the Modeler
- **Decision Instances** are executed using the Zeebe Engine
- **Decision Instances** can be monitored using Operate

Create a Decision Requirements Diagram (DRD)

Create a **Decision Requirements Diagram** to represent the *Request Car Insurance* decision structure

Create a Decision Table

Create a **Decision Table** to represent the *Request Car Insurance* business rules

Configure Business Rules

The request will be accepted or rejected based upon the experience of the **Driver**.

Configure Business Rules

Add **Business Rules** to the **Decision Table** in order to meet the **Acceptance Criteria**

Replace Process Logic with Business Rules

The current version of the *Request Car Insurance* process has implemented the decision logic using BPMN constructs. A more flexible and powerful approach would be to use a **Business Rule Task** which references the newly created **Decision Table**.

Add a Business Rule Task

Add a **Business Rule Task** to the process definition to encapsulate the decision logic

Link the Business Rule Task

Link the **Business Rule Task** with the previously created **Decision Table**

Update Business Rules

The **Business Rules** that have been configured in the **Decision Table** should be expanded to cover the type of car covered by the request. Feedback should also be provided to the **Insurance Manager** on the reason for the decision.

Add Decision Table Input

Add a new input to the **Decision Table** to represent the *Car Type*

Add Decision Table Output

Add a new output to the **Decision Table** to represent the *Risk Description*

Development Plan

Now that we have identified the components and features of Camunda which will be used to implement our solution, we can create a plan of the work that needs to be performed and the tools that will be required.

- **Create a Decision Requirements Diagram (DRD)**
  - This will contain our **Decision Table** and will be configured using the Modeler
- **Configure a Decision Table**
  - This will contain our **Business Rules** and will be configured using the Modeler
- **Deploy a Decision Requirements Diagram (DRD)**
  - This will be performed using the Modeler
- **Import Process Definition**
  - This will allow us to modify the existing process definition and will be performed using the Modeler
- **Update the Process Definition**
  - This will be performed using the Modeler
- **Deploy the Process Definition**
  - This will be performed using the Modeler
- **Start a Process Instance**
  - This will be performed from the Modeler
- **Monitor the Process Definition**
  - This will be performed from Operate
- **Update the Decision Requirements Diagram (DRD)**

- This will be performed using the Modeler

Import Process

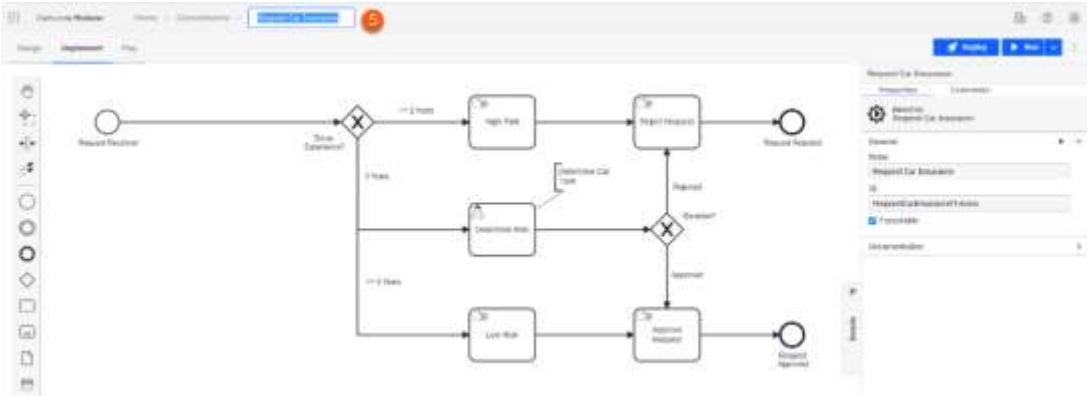
We will now import the current version of the *Request Car Insurance* process into our **Project**.

Import Process Definition

Import the existing **Process Definition** by following the steps below:

1. Download the *Request Car Insurance* process from this GitHub Repository and save it to your local filesystem
2. From the *Camundanzia* project in the **Modeler**, click the **Create new file** button and then **Upload files** from the menu that appears

3. From the dialog box that appears, select the `requestCarInsurance.bpmn` that you just downloaded. This will import the existing **Process Definition** into our project.
4. Double click on the process that you just imported to open it in the Modeler.
5. Click on the **Diagram Name** and then **Edit Name** from the top menu to change the name of the BPMN Diagram. For this course we will use *Request Car Insurance*.



Imported a Process Definition

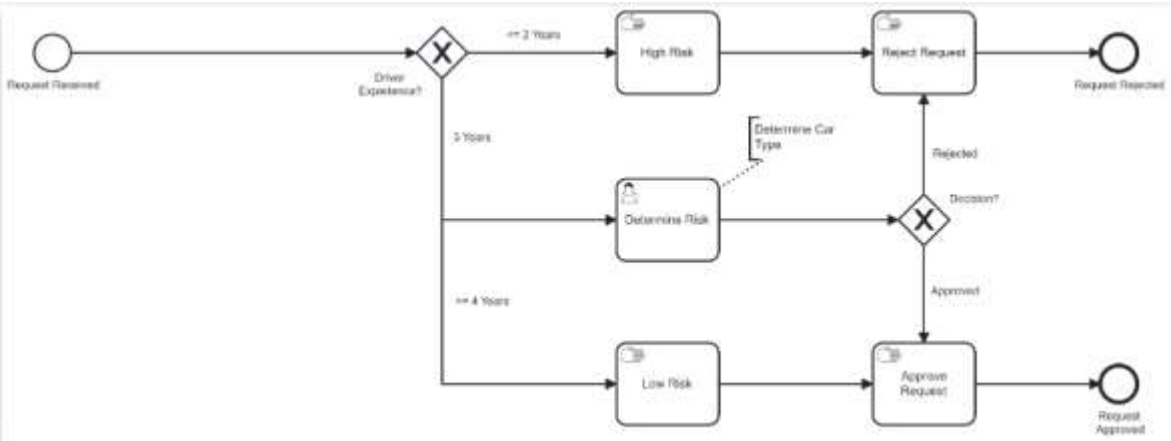
Congratulations! You have now imported the existing **Process Definition** into your project.

Review Process

Let's take a look at the current version of the Camundanzia *Request Car Insurance* process.

Request Car Insurance

The process starts when a request for Car Insurance is received from a customer.



1. The Exclusive Gateway (XOR) evaluates the **Driver's** experience; the appropriate sequence flow is then selected
2. If the **Driver's** experience is less than or equal to 2 years, the risk is considered to be *High* and the request will be rejected
3. If the **Driver's** experience is greater than 4 years, the risk is considered to be *Low* and the request will be approved
4. If the **Driver's** experience is 3 years, there is a need for a human decision. The **User Task** named *Determine Risk* will be performed by an **Insurance Specialist** to determine whether the request should be *Approved* or *Rejected*.

Manual Tasks

This process has been simplified to allow us to focus on **Decision Automation**. The **Manual Tasks** in the process would normally be implemented as **Service Tasks**. However, the use of **Manual Tasks** allows us to work through this course without the need to provide an implementation; the tasks are simply skipped.

Areas for Improvement

The current version of the *Request Car Insurance* works well and meets the current requirements of Camundanzia.

However, there is a flaw. What would happen if the **Insurance Manager** changes the **Policy Rules** so that **Drivers** with 3 years of experience are now rejected?

What happens when the Policy Rules change?

A **Process Developer** would need to perform the following steps to address the new requirement:

1. Update the **Process Definition**; the conditions on each sequence flow would need to be updated to reflect the new rules
2. Deploy the **Process Definition**; this would create a new version of the process with the new rules

Business rules in organizations tend to change frequently, whereas the underlying processes tend to change infrequently. For that reason, a better approach would be to separate the **Business Rules** from the **Business Process** and manage them independently; this is where DMN can help.

We can replace the business rules that are encapsulated in the exclusive gateway and sequence flows with a **Business Rule Task** that is connected to a **Decision Table**.

Deploy Process

We will now need to deploy our **Process Definition** to the previously created **Cluster** so that we can start a **Process Instance**.

Deploy Process Definition

Deploy the **Process Definition** by following the steps below:

What is a Deployment?

A **Deployment** allows the **Process Definition** to be executed by the **Zeebe Engine**.

1. Select the **Implement** mode if it has not already been selected
2. Click on the **Deploy** button in the top right corner of the **Modeler**
3. Ensure that the *Camundanzia* **Cluster** is selected from the dialog box that appears.
4. Click the **Deploy** button to deploy the **Process Definition** to the **Cluster**

5. A **Diagram Deployed** message will be displayed at the bottom of the **Modeler** to show that this was successful.

View the Deployment in Operate

Next, we can take a look into **Operate** to confirm that the deployment has been successful.

What is Operate?

Operate is a tool for monitoring and troubleshooting process instances running in the **Zeebe Engine**.

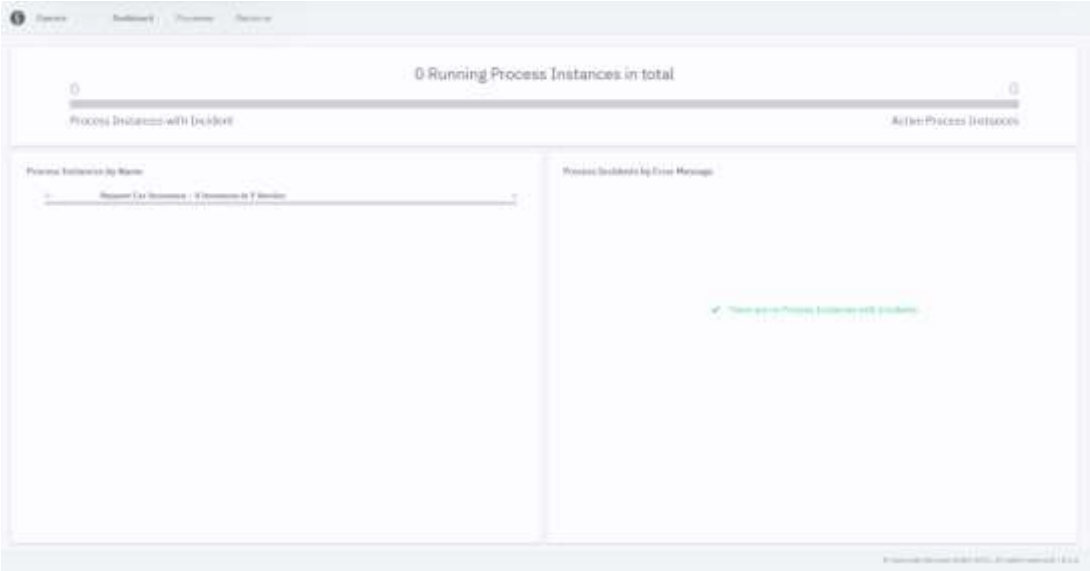
In addition to providing visibility into active and completed process instances, Operate also makes it possible to carry out key operations such as resolving incidents, and updating process instance variables.

Launch Operate

Launch **Operate** by following the steps below:

1. From the **Console**, select **Clusters**.
2. Select *Camundanzia* from the **Clusters** page.
3. Click the **Launch** link next to **Operate** in the **Cluster Details** page

4. After a few seconds, the **Operate** Dashboard will be displayed.
5. You will also be able to see that the *Request Car Insurance* process is visible in the Dashboard.



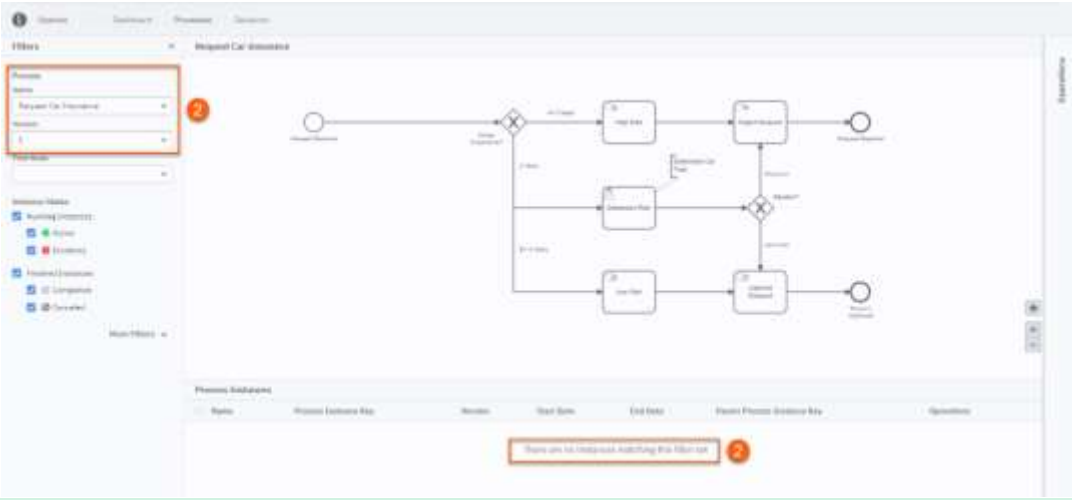
View Process Instances

We can now view the process instances in **Operate** by following the steps below:

What is a Process Instance?

One execution of a **Process Definition** is called a **Process Instance**. Many **Process Instances** can be launched from one **Process Definition**.

1. Click on *Request Car Insurance* from the **Process Instances by Name** panel on the left-hand side of the screen
2. **Operate** will not show any active process instances at this point. However, we can see that version 1 of our *Request Car Insurance* process has been successfully deployed.



Deployed a Process Definition

Congratulations! You have now deployed the *Request Car Insurance* **Process Definition** into the **Zeebe Engine**.

Create Decision Requirements Diagram (DRD)

We will now create a new **Decision Requirements Diagram (DRD)** in our Project which will be used to model the *Request Car Insurance* business rules. **Decision Tables** are added to a **Decision Requirements Diagram (DRD)** in Camunda.

What is a Decision Requirements Diagram (DRD)?

A Decision Requirements Diagram (DRD) is a DMN notation that allows for the definition of complex decisions. DRDs contain:

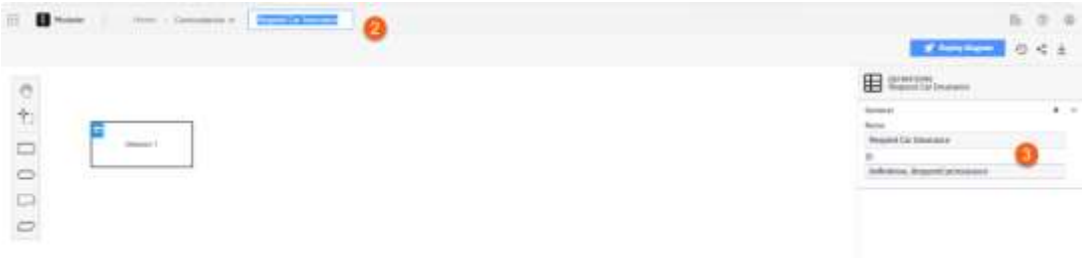
- **Decisions** - The act of determining an output value from a number of input values, using logic definition. This decision logic is what you can express in **Decision Tables**
- **Input Data** - The input data that you “feed” into your decision logic in order to determine the output value.
- **Relations between Decisions** - You can connect decisions with arrows and therefore indicate which decision output will be considered as an input for another decision.

Create Decision Requirements Diagram (DRD)

Create a new Decision Requirements Diagram by following the steps below:

1. From the *Camundanzia* project in the **Modeler**, click the **New** button and then **DMN Diagram** from the menu that appears

2. This will open the Modeler with a new **DMN Diagram**. Click on the **Diagram Name** and then **Edit Name** from the menu that appears to change the name of the DMN Diagram. For this course we will use *Request Car Insurance*.



3. In the properties panel on the right-hand side of the screen, set the **Name** and **ID** as follows:

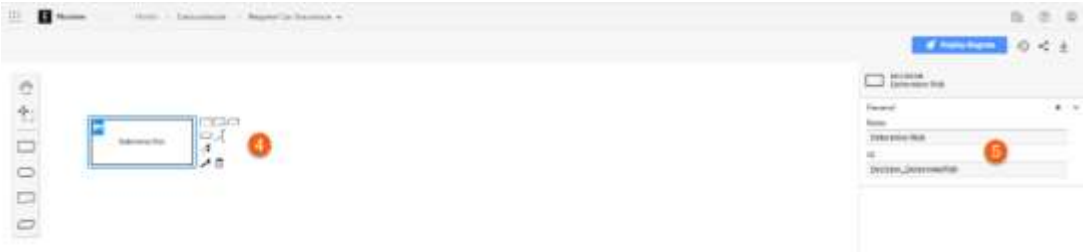
Name	Value
Name	Request Car Insurance
ID	Definitions_RequestCarInsurance

Naming Conventions for IDs

Define developer-friendly and business-relevant IDs for the process itself, as well as all activities, messages, and errors. Also consider events, gateways, and the sequence flows that carry conditional expressions. Even though IDs are just identifiers, keep in mind that they will show up regularly on the technical level. Meaningful IDs will help a lot.

4. Click on the border of the rectangle that has been created (named Decision 1)
5. In the properties panel on the right-hand side of the screen, set the **Name** and **ID** as follows:

Name	Value
Name	Determine Risk
ID	Decision_DetermineRisk



Decision Requirements Diagram (DRD) created

Congratulations! You have now created a Decision Requirements Diagram (DRD) which will capture the business rules of the *Request Car Insurance* process.

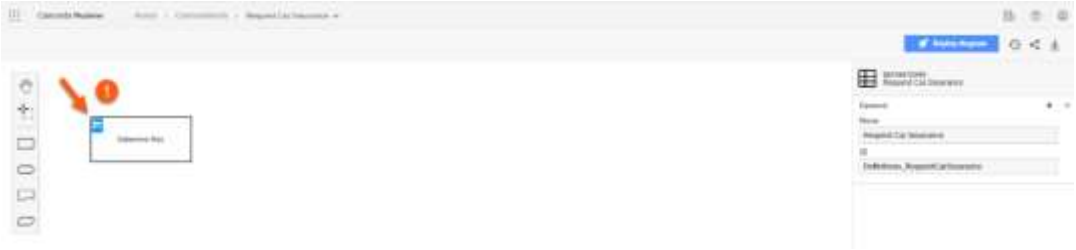
Create Decision Table

We will now configure our **Decision Table** with the business rules of the *Request Car Insurance* process.

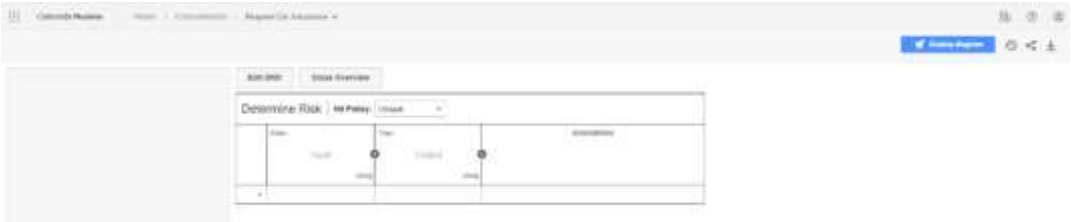
Configure Decision Table

Configure the **Decision Table** by following the steps below:

1. Click on the **Blue Icon** in the top-left hand corner of the *Determine Risk* Decision Table.

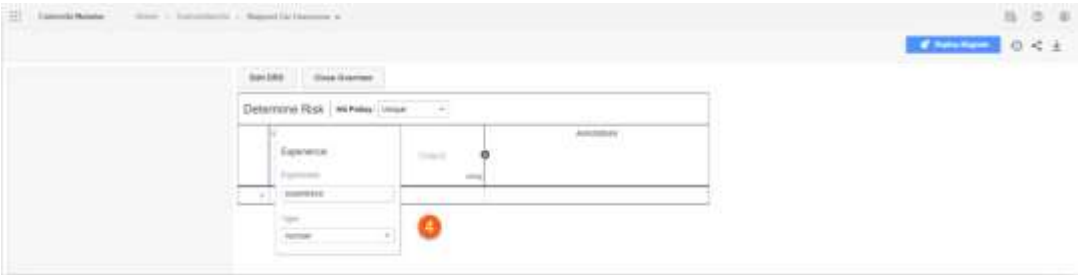


2. This will open the **Decision Table Editor** in the Modeler



3. Let's rename the columns so that they reflect the *Request Car Insurance* business rules.
4. Double click on the **Input** column and configure it as follows:

Name	Value
Name	Experience
Expression	experience
Type	number



Column Naming

The **Name** of the column is used as a label only. The **Expression** field will contain the value of the **Process Variable** that is passed from the **Process Instance**.

Data Types

Camunda supports several different DMN data types such as `number`, `string` and `boolean`; a complete list can be found in the [documentation](#).

5. Double click on the **Output** column and configure it as follows:

Name	Value
Name	Code
Output Name	code
Type	string
Predefined Values	"red", "yellow", "green"

Column Naming

The **Name** of the column is used as a label only. The **Output Name** field will populate the corresponding **Process Variable** in the **Process Instance**.

Predefined Values

Predefined Values are useful if there is a clear set of values that the specific input or output can have. By filling the rules, the predefined values can be selected from a list to prevent errors. However, it is still possible to add other values directly to the rules of the table later.

Remember to hit enter after typing the predefined values; they will be lost when the dialog box closes otherwise.

Configure Business Rules

We can now configure the **Business Rules** for the **Decision Table** by following the steps below:

1. Use the **+** icon to add three new **Rules** to the **Decision Table**. These **Rules** correspond to the **Business Requirements** that we identified earlier

Rules

Each line in a **Decision Table** represents a **Rule**. A **Rule** consists of an **Input Expression** and an **Output Expression**. If the **Input Expression** returns **true**, then the corresponding output will be returned.

2. Populate each of the rows with the following rules

Experience (Years)	Code	Annotations
<=2.0	"red"	
]2.0 .. 3.0]	"yellow"	Determine Risk Manually
>3.0	"green"	

Friendly Enough Expression Language (FEEL)

The rules in this table are defined using Friendly Enough Expression Language (FEEL)

3. Your **Decision Table** should look similar to the below when you have finished

FEEL Comparisons

Comparisons can be added with the following operators =, < , >, <=, >=.

What happened to the square bracket?

The second rule uses the **Interval Notation** with square brackets. It basically means *all the numbers between 2.0 and 3.0*. Since the first square bracket is inverted, it means that the first number is excluded from the interval. Alternatively, we could have also entered 3.0 here but using the interval provides more flexibility for the future.

Decision Table configured

Congratulations! You have now configured a Decision Table with the business rules of the *Request Car Insurance* process.

Deploy Decision Requirements Diagram (DRD)

We will now need to deploy our **Decision Requirements Diagram** to the previously created **Cluster** so that we can start a **Decision Instance**.

Deploy Decision Requirements Diagram

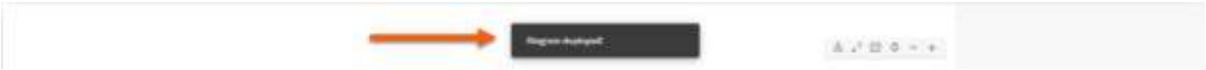
Deploy the **Decision Requirements Diagram** by following the steps below:

What is a Deployment?

A **Deployment** allows the **Decision Table** to be executed by the **Zeebe Engine**.

1. Click on the **Deploy Diagram** button in the top right corner of the **Modeler**
2. Ensure that the *Camundanzia* **Cluster** is selected from the dialog box that appears.
3. Click the **Deploy** button to deploy the **Decision Requirements Diagram** to the **Cluster**

4. A **Diagram Deployed** message will be displayed at the bottom of the **Modeler** to show that this was successful.



View the Deployment in Operate

Next, we can take a look into **Operate** to confirm that the deployment has been successful.

What is Operate?

Operate is a tool for monitoring and troubleshooting decision instances running in the **Zeebe Engine**.

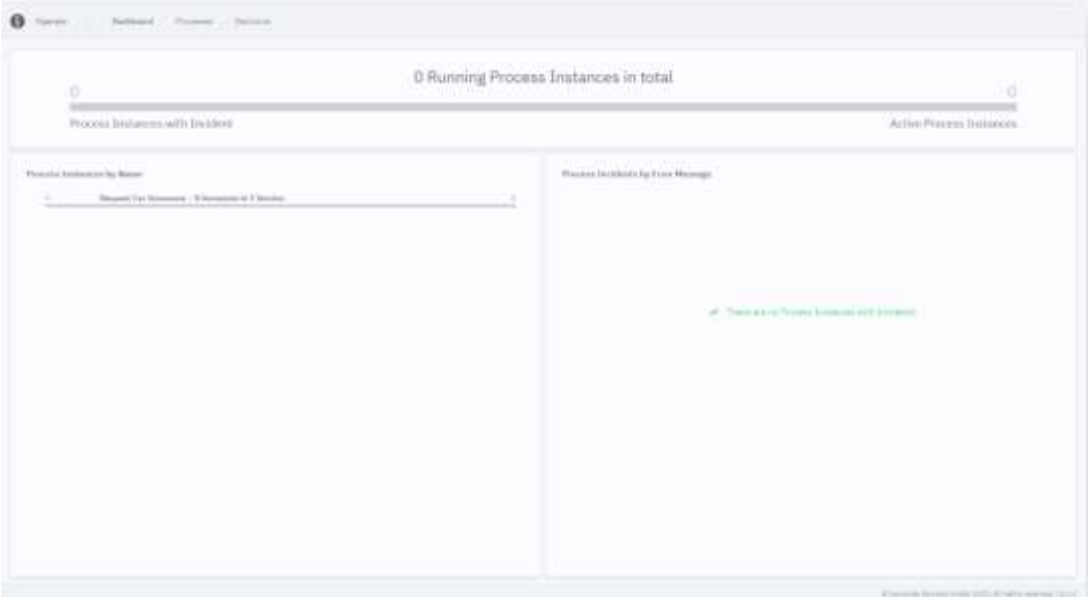
In addition to providing visibility into active and completed decision instances, Operate also makes it possible to carry out key operations such as resolving incidents, and updating process instance variables.

Launch Operate

Launch **Operate** by following the steps below:

1. From the **Console**, select **Clusters**.
2. Select *Camundanzia* from the **Clusters** page.
3. Click the **Launch** link next to **Operate** in the **Cluster Details** page

4. After a few seconds, the **Operate** Dashboard will be displayed.
5. You will also be able to see that the *Request Car Insurance* process is visible in the Dashboard.



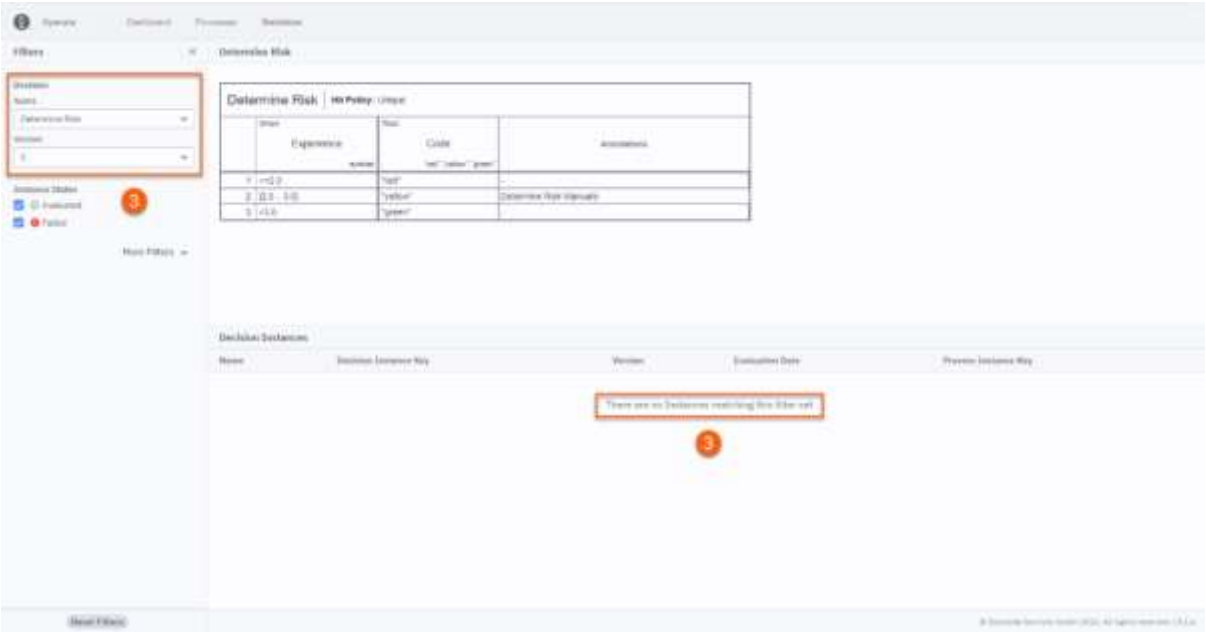
View Decision Instances

We can now view the decision instances in **Operate** by following the steps below:

What is a Decision Instance?

One execution of a **Decision Table** is called a **Decision Instance**. Many **Decision Instances** can be launched from one **Decision Table**.

1. Click on *Request Car Insurance* from the **Process Instances by Name** panel on the left-hand side of the screen
2. Click on **Decisions** from the top-menu bar
3. **Operate** will not show any active decision instances at this point. However, we can see that version 1 of our *Determine Risk* decision table has been successfully deployed.



Deployed a Decision Requirements Diagram

Congratulations! You have now deployed the *Request Car Insurance* **Decision Requirements Diagram** into the **Zeebe Engine**.

Configure Business Rule Task

We now need to update our *Request Car Insurance* process to use the *Determine Risk* **Decision Table** that we have just deployed. This will be achieved by using a **Business Rule Task**.

What is a Business Rule Task?

A **Business Rule Task** is used to model the evaluation of a business rule; for example, a decision modeled in Decision Model and Notation (DMN).

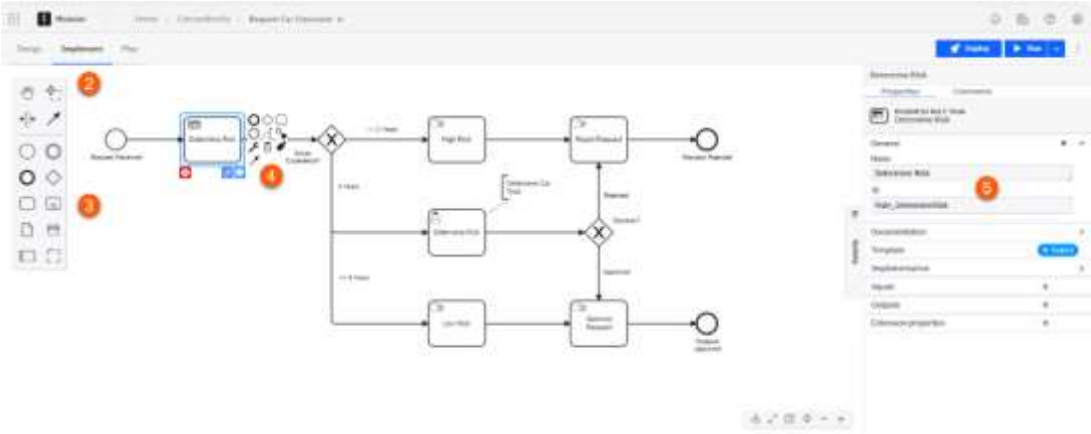
Add Business Rule Task

Add the **Business Rule Task** by following the steps below:

1. In the Modeler, open the *Request Car Insurance* process that we imported earlier
2. Select the **Implement** mode if it has not already been selected
3. Drag the **Create Task** icon from the palette and drop it on the sequence flow between the **Start Event** and the **Gateway**
4. Use the **Wrench Tool** from the context menu to change the newly added task to a **Business Rule Task**
5. In the properties panel on the right-hand side of the screen, select **General** and then set the **Name** and **ID** as follows:

Name	Value
Name	Determine Risk

Name	Value
ID	Rule_DetermineRisk



Process Validation Warning

6. You can ignore the warning about `A <Business Rule Task> must have a defined <Implementation>`; we will fix this in the next section.

Configure Business Rule Task

Configure the **Business Rule Task** to use the *Determine Risk* **Decision Table** by following the steps below:

1. Select the **Business Rule Task** and from the properties panel on the right-hand side of the screen select **DMN Decision** for the **Implementation**
2. Click on **Called Decision** and provide the ID of the *Determine Risk* **Decision Table** that was created earlier.

Name	Value
Decision ID	Decision_DetermineRisk
Result Variable	risk

Result Variable

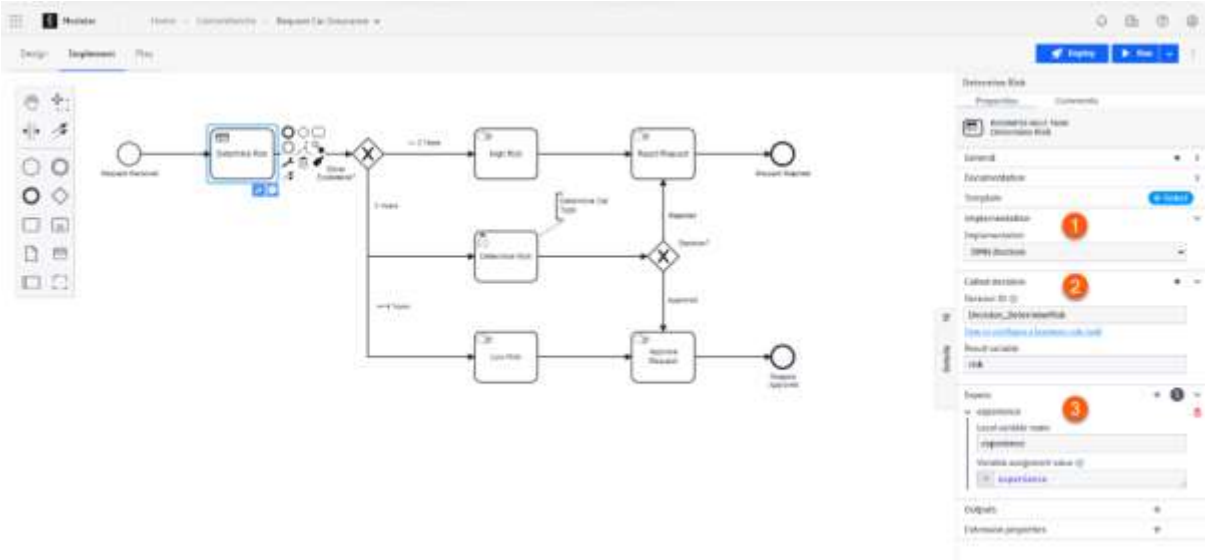
The **Result Variable** is the name of the **Process Variable** which will be populated with the results of the **Decision Table**.

3. Click on **Inputs** and provide the inputs to the *Determine Risk* **Decision Table** that was created earlier.

Name	Value
Local Variable Name	experience
Variable Assignment Value	experience

Inputs

The **Input** is the name of the **Process Variable** which will be passed into the input of the **Decision Table**.



Business Rule Task configured

Congratulations! You have now configured a **Business Rule Task** to call a **Decision Table** from the *Request Car Insurance* process.

Configure Gateway

We now need to update our *Request Car Insurance* process to use the outputs from the *Determine Risk* **Decision Table**.

The *Driver Experience* gateway is still evaluating conditions for the next sequence flow based upon the experience of **Driver**. This needs to be modified so that it evaluates conditions based upon *Risk*.

Separation of Business Rules and Business Process

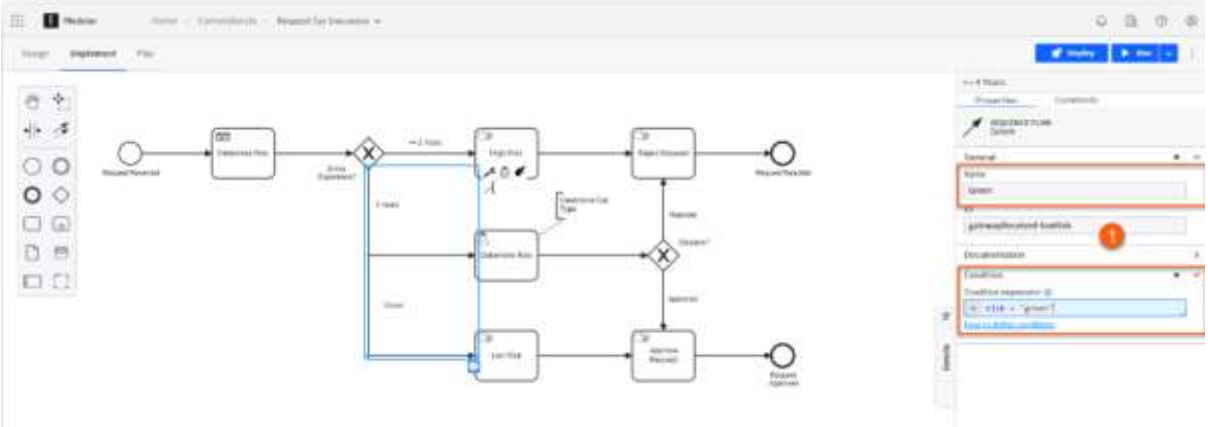
The advantage of making this change is that the *Risk* is now stable; it will only be Red, Yellow, or Green. If the **Business Rules** change again, we will only need to update the **Decision Table** and not the **Business Process**.

Modify Gateway Conditions

Modify the exclusive gateway conditions by following the steps below:

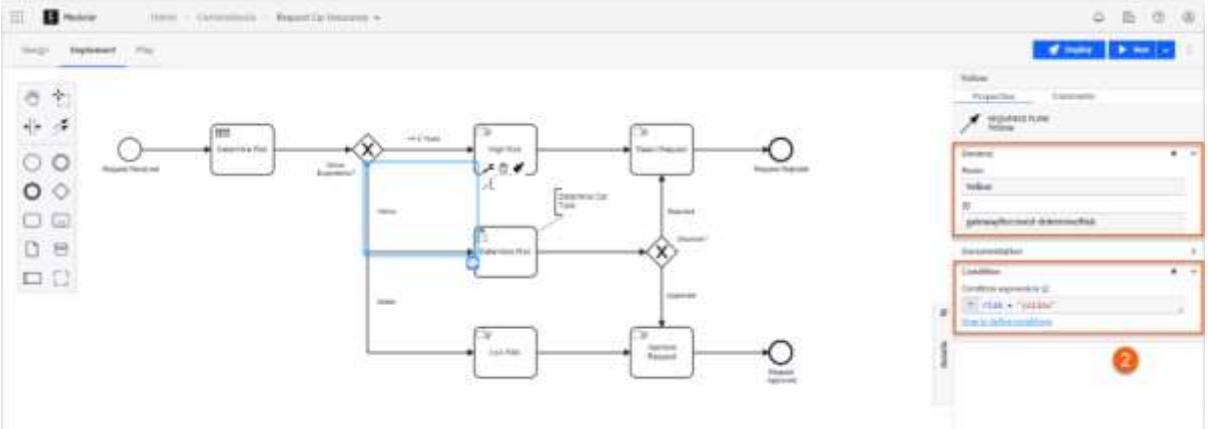
1. Select the sequence flow leading to *Low Risk* (labelled >- 4 Years) and modify the **Name** and **Condition Expression** as below:

Name	Value
Name	Green
Condition Expression	risk = "green"

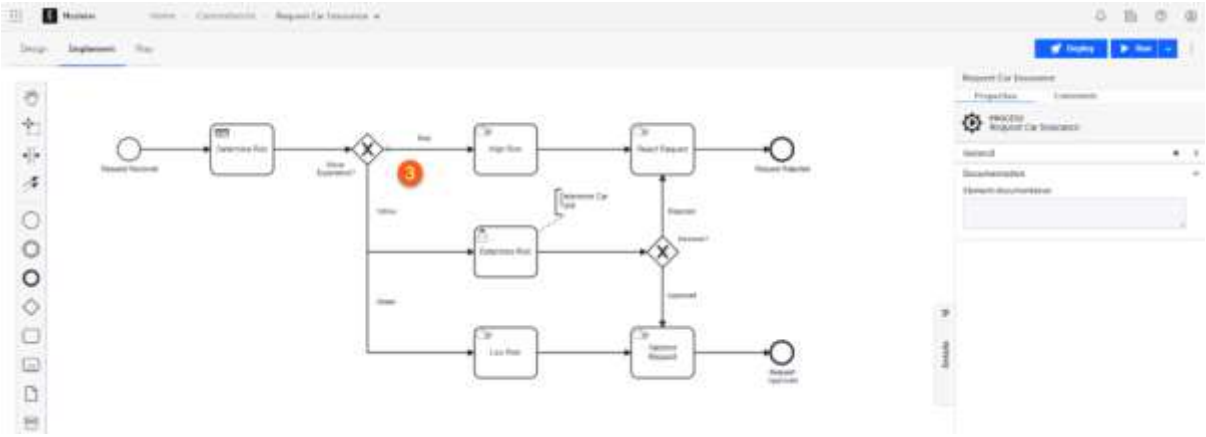


2. Repeat the same steps for the sequence flow leading to *Determine Risk* (labelled 3 Years) and modify the **Name** and **Condition Expression** as below:

Name	Value
Name	Yellow
Condition Expression	risk = "yellow"



3. We will configure the final sequence flow leading to *High Risk* as the **Default Flow**. Set the **Name** of the sequence flow to `Red` and clear the **Condition** field. Finally, click the **Wrench Icon** on the sequence flow and select **Default Flow**.

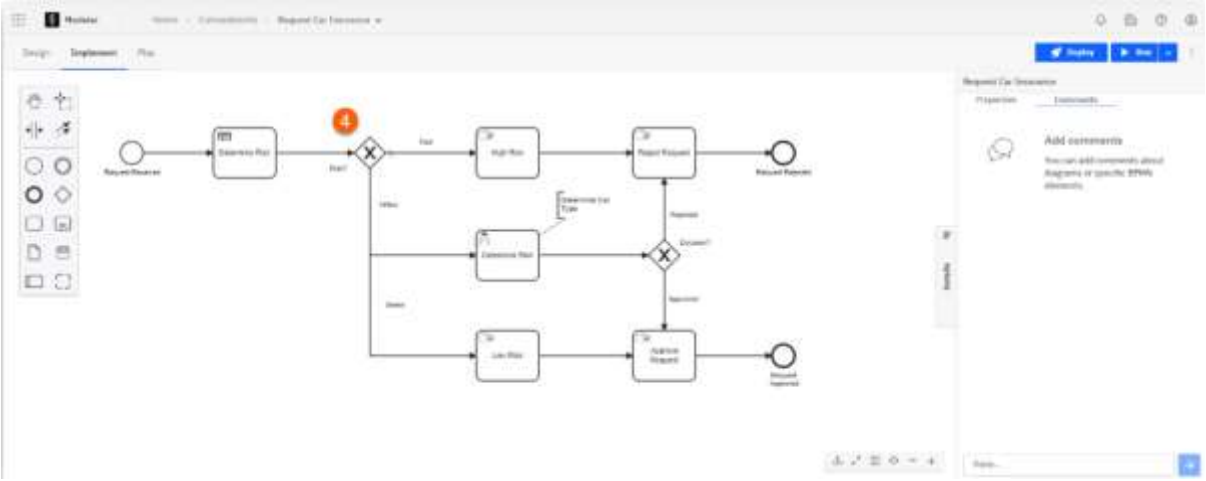


Default Flow

The **Default Flow** does not require a condition; this flow will be selected if no other conditions evaluate to true. A `\` is added to the sequence flow to denote that it is the default.

4. Finally, update the properties of the exclusive gateway as follows:

Name	Value
Name	Risk?
ID	gatewayRisk



Updated Process Routing

Congratulations! You have now updated the process routing so that it uses **Business Rules** rather than **Process Logic**

Deploy Process

We will now need to deploy our updated **Process Definition** to the previously created **Cluster** so that we can start a **Process Instance**.

Deploy Process Definition

Deploy the **Process Definition** by following the steps below:

1. Select the **Implement** mode if it has not already been selected
2. Click on the **Deploy** button in the top right corner of the **Modeler**
3. Ensure that the *Camundanzia* **Cluster** is selected from the dialog box that appears.
4. Note that you are also reminded to deploy your **Decision Requirements Diagram** as it is now linked to the **Process Definition**
5. Click the **Deploy** button to deploy the **Process Definition** to the **Cluster**

6. A **Diagram Deployed** message will be displayed at the bottom of the **Modeler** to show that this was successful.

View the Deployment in Operate

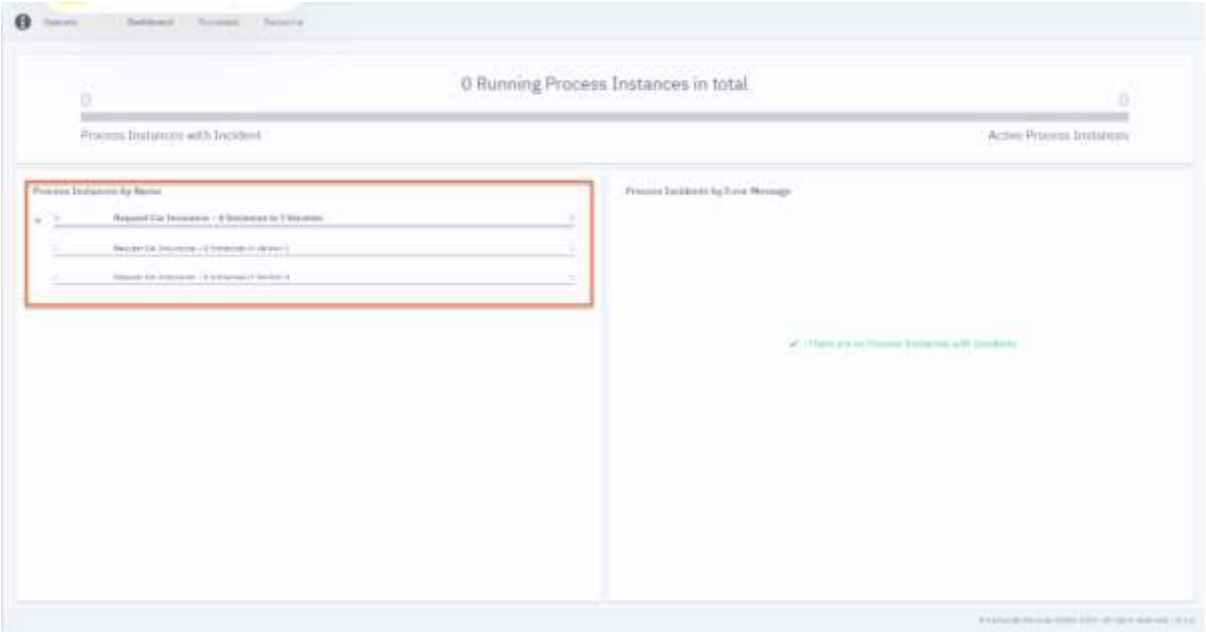
Next, we can take a look into **Operate** to confirm that the deployment has been successful.

Launch Operate

Launch **Operate** by following the steps below:

1. From the **Console**, select **Clusters**.
2. Select *Camundanzia* from the **Clusters** page.
3. Click the **Launch** link next to **Operate** in the **Cluster Details** page

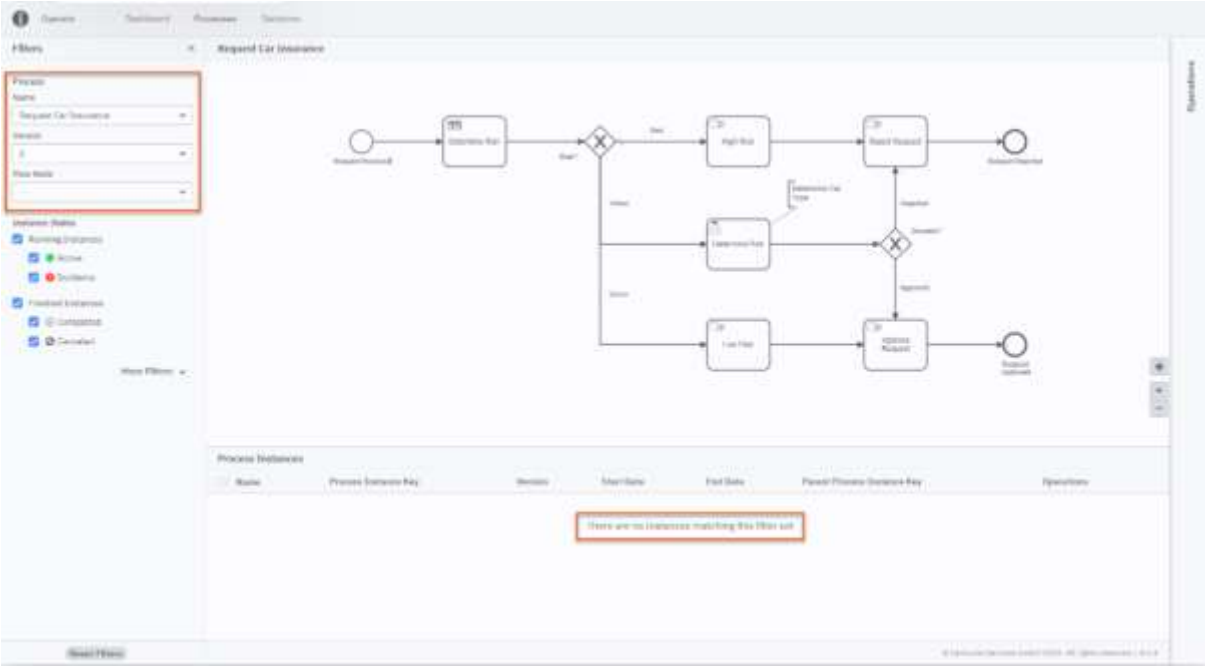
4. After a few seconds, the **Operate** Dashboard will be displayed.
5. You will also be able to see that two versions of the *Request Car Insurance* process are visible in the Dashboard.



View Process Instances

We can now view the process instances in **Operate** by following the steps below:

1. Click on *Request Car Insurance* from the **Process Instances by Name** panel on the left-hand side of the screen
2. **Operate** will not show any active process instances at this point. However, we can see that version 2 of our *Request Car Insurance* process has been successfully deployed.



Deployed a Process Definition

Congratulations! You have now deployed the updated *Request Car Insurance* **Process Definition** into the **Zeebe Engine**.

Start Process

We will now need to execute our **Process Definition** and **Decision Table** for the first time; this is also known as starting a **Process Instance** and starting a **Decision Instance**.

Start a Process Instance

Start an instance of the *Request Car Insurance* by following the steps below:

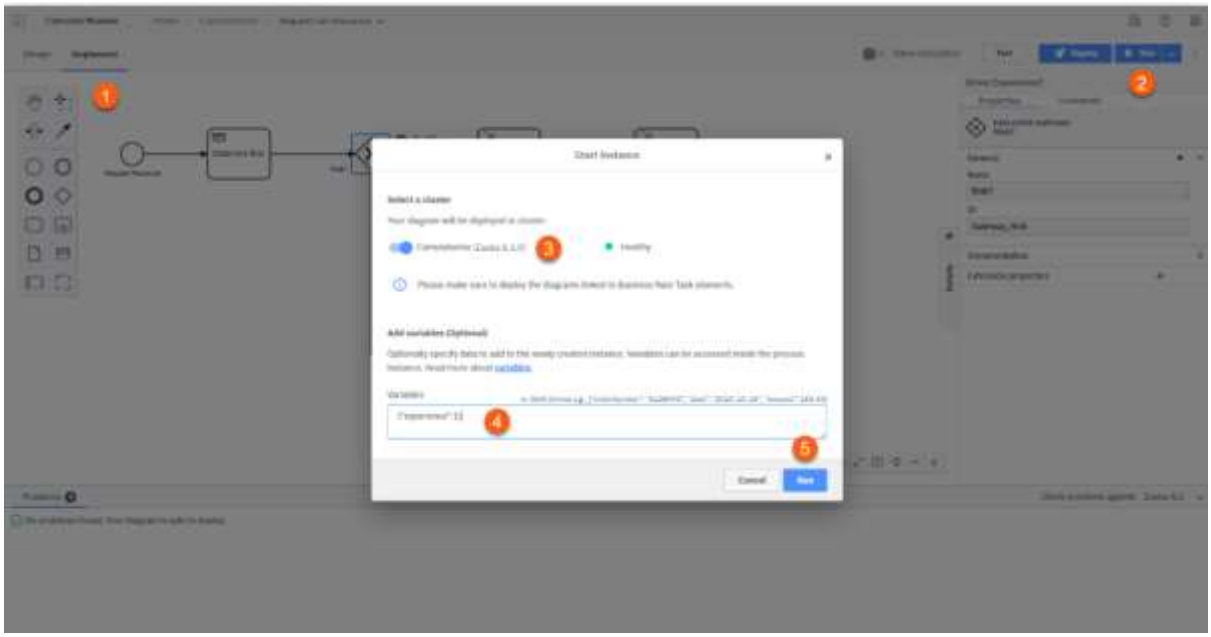
1. Select the **Implement** mode if it has not already been selected
2. Click on the **Run** button in the top right corner of the **Modeler**
3. Ensure that the *Camundanzia* **Cluster** is selected from the dialog box that appears.
4. We need to start the process with a value for the driver's experience; enter the following into the **Add Variables** section

```
{"experience":1}
```

Process Variables passed to Decision Table

The **Process Instance** will pass the *experience* as an input to the *Determine Risk* **Decision Table**. The **Decision Instance** will then pass the *risk* back to the **Process Instance**.

5. Click the **Run** button to start a new instance of *Request Car Insurance* using the *Camundanzia* **Cluster**

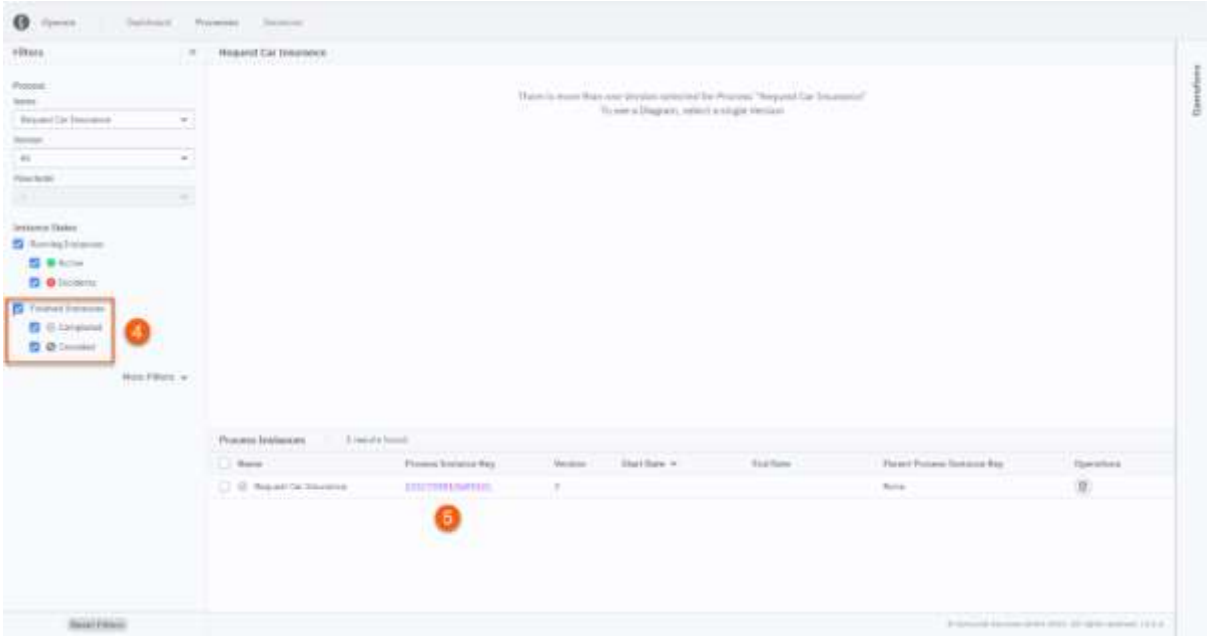


6. An **Instance Started!** message will be displayed at the bottom of the **Modeler** to show that this was successful.

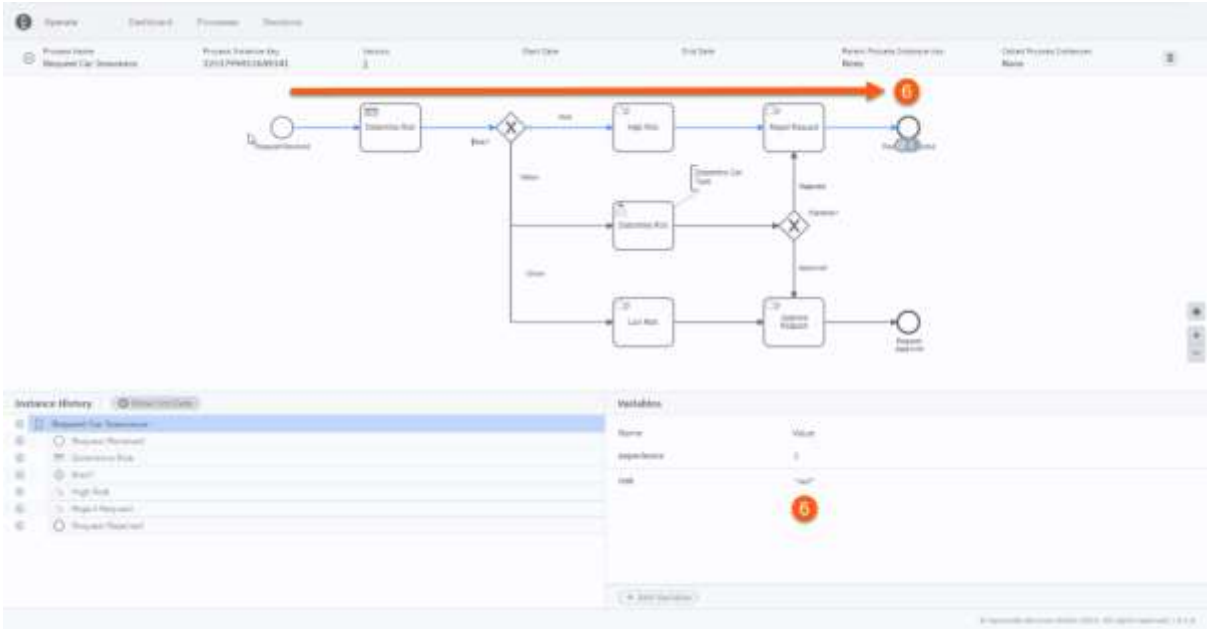
View Process Instances

We can now view the **Process Instances** in **Operate** by following the steps below:

1. Launch **Operate** by following the steps in the previous lesson. Alternatively, refresh the window if you still have **Operate** open.
2. Click on *Request Car Insurance* from the **Process Instances by Name** panel on the left-hand side of the screen
3. **Operate** will now show an empty screen as there are no running instances of our *Request Car Insurance* process
4. Check the **Finished Instances** filter on the left hand side of the screen to show completed process instances



5. Click on the **Process Instance Key** which now appears in the bottom part of the screen; this will open up the **Process Instance View**

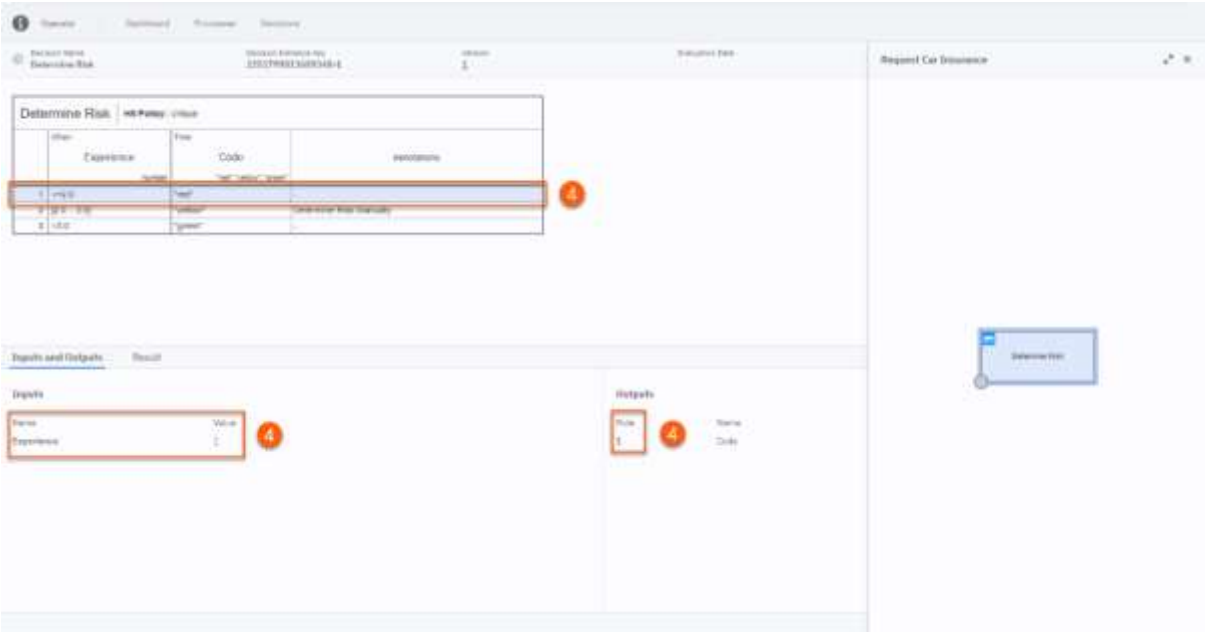


6. We can now see that the process completed with the *Application Rejected* event, that the *risk* was set to *red* and that the process instance followed the expected path

View Decision Instances

We can now view the **Decision Instances** in **Operate** by following the steps below:

1. Click on **Decisions** from the top menu bar
2. Select *Determine Risk* from the **Decision Name** on the left hand side of the screen to show the **Decision Table** and **Decision Instances**
3. Click on the **Decision Instance Key** which now appears in the bottom part of the screen; this will open up the **Decision Instance View**



4. We can now see the **Inputs**, **Outputs** and **Results** of the **Decision Table**. As expected, Rule 1 was triggered and the value returned was *red*.

Started a Process Instance and Decision Instance

Congratulations! You have now started an instance of the *Request Car Insurance* process and an instance of the *Determine Risk* decision table.

Modify Decision Table

Our initial version of the **Decision Table** evaluates risk solely using **Driver** experience.

This is a good starting point but the business requirements have now changed. Now, the *Car Type* also needs to be evaluated before a request can be accepted; the risk will be higher for cars that are deemed to be *Luxury*.

Modify the Decision Table

Add Input (Car Type)

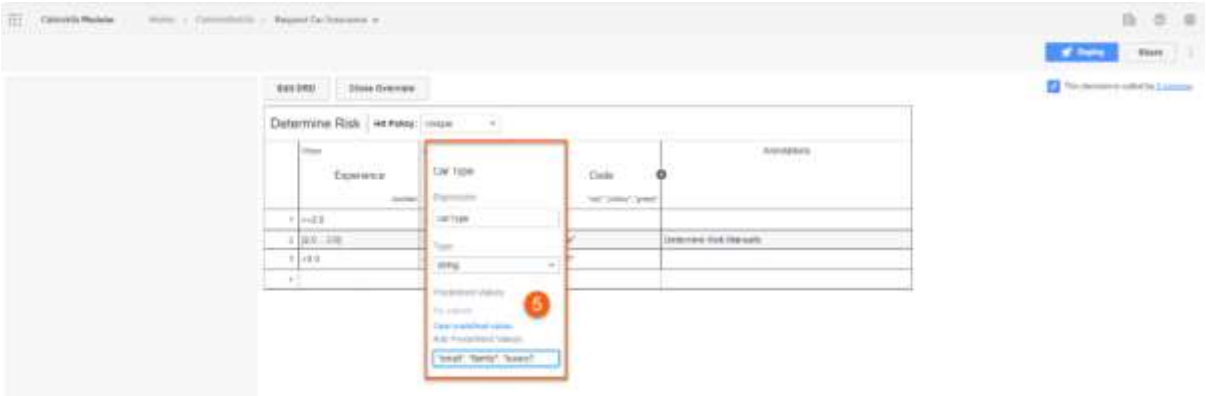
Modify the **Decision Table** by following the steps below:

1. From the *Camundanzia* project in the **Modeler**, click on the *Request Car Insurance* **Decision Requirements Diagram**

2. Click on the **Blue Icon** in the top-left hand corner of the *Determine Risk* Decision Table.
3. This will open the **Decision Table Editor** in the Modeler

4. Click the **+** icon between the **Experience (Years)** and **Code** column to add a new **Input Column**
5. Double click on the **Input** column and configure it as follows:

Name	**Value**
Name	Car Type
Expression	carType
Type	string
Predefined Values	"small", "family", "luxury"



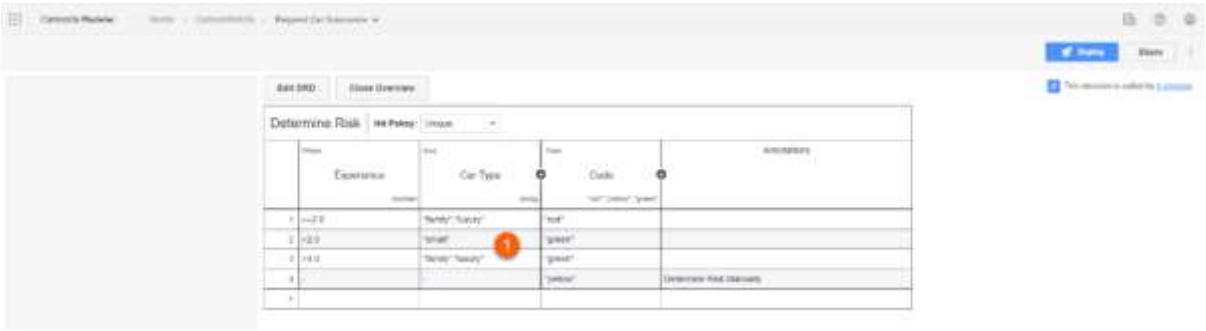
Modify Business Rules

You now need to change the business rules in the **Decision Table** to take into account the *Car Type*. Inexperienced drivers with *Luxury* cars will result in a *Red* risk. *Small* cars will only require 2 years of experience and *Family* cars will require more than 4 years of experience. All other cases will require a manual determination.

Modify the **Decision Table** by following the steps below:

1. Populate each of the rows with the following rules

Experience (Years)	Car Type	Code	Annotations
<=2.0	"family","luxury"	"red"	
>2.0	"small"	"green"	
>4.0	"family","luxury"	"green"	
-	-	"yellow"	Determine Risk Manually



Empty Rows

The **-** symbol in the fourth rule represents a wildcard. This does not need to be explicitly added as the Modeler will generate this automatically when you leave the cell empty.

The fourth rule effectively means that the risk will be *Yellow* regardless of the *Experience* and *Car Type*; this provides us with a default behavior if a match is not found in the other rules.

Change Hit Policy

We haven't configured the **Hit Policy** yet. Up until now, we have used the default, **Unique**, but there are several other options available.

What is a Hit Policy?

Hit policies describe different ways (standardized by DMN) to evaluate the rules contained in a decision table. Different hit policies do not only lead to different results, but typically also require different modes of thinking and reasoning about the meaning of the entire decision table.

Unique Hit Policy

The **Unique** hit policy returns a single result and as the name suggests, two rules cannot overlap and evaluate to **True** at the same time.

In our decision table, the fourth rule overlaps with the other rules and therefore prevents us from using the **Unique** hit policy.

However, we can change the hit policy to **First** which evaluates the rules from top to bottom and returns the first rule that evaluates as **True**. Our fourth rule then becomes the default and is returned when all previous rules have evaluated to false.

1. Change the hit policy for decision table to **First** using the steps below:

Add Output (Description)

Our decision table only returns one output, the *Risk Code*. We can also a return a *Risk Description* which can be used by our process.

Modify the **Decision Table** by following the steps below:

1. Click the **+** icon between the **Code** and **Annotations** column to add a new **Output Column**
2. Double click on the **Output** column and configure it as follows:

Name	**Value**
Name	Description
Expression	description
Type	string

3. Populate each of the rows with the following rules

Experience (Years)	**Car Type**	**Code**	**Description**	**Annotations**
<=2.0	"family","luxury"	"red"	"Too Risky"	
>2.0	"small"	"green"	"Small Car, No Risk"	
>4.0	"family","luxury"	"green"	"Experienced Driver, No Risk"	

Experience (Years)	Car Type	Code	Description	Annotations
-	-	"yellow"	"Manual Determination"	Determine Risk Manually

Deploy Decision Requirements Diagram

Deploy the **Decision Requirements Diagram** by following the steps below:

1. Click on the **Deploy Diagram** button in the top right corner of the **Modeler**
2. Ensure that the *Camundanzia* **Cluster** is selected from the dialog box that appears.
3. Click the **Deploy** button to deploy the **Decision Requirements Diagram** to the **Cluster**
4. A **Diagram Deployed** message will be displayed at the bottom of the **Modeler** to show that this was successful.

Deployed Decision Table

Congratulations! You have now deployed the updated *Determine Risk* **Decision Table** into the **Zeebe Engine**.

Modify Process

As we have modified the structure of the **Decision Table**, we will also need to make the corresponding modifications to the **Process Definition**. We won't be adding new elements, simply tweaking the existing configuration.

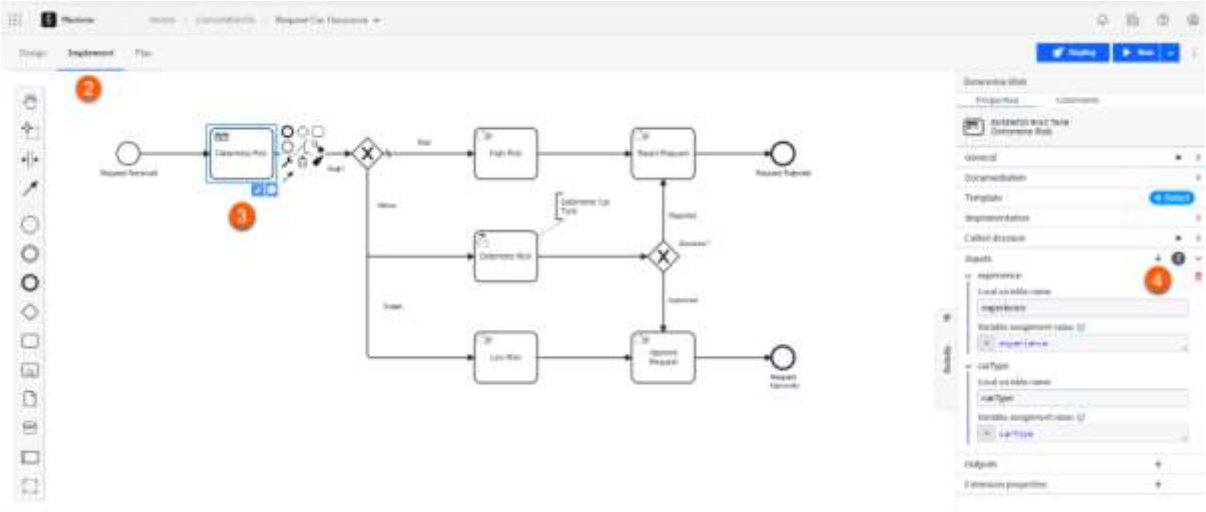
Modify Business Rule Task

Modify the **Business Rule Task** by following the steps below:

1. In the Modeler, open the *Request Car Insurance* process that we imported earlier
2. Select the **Implement** mode if it has not already been selected
3. Click on the *Determine Risk* task
4. Add a new **Input Variable** by clicking on the **+** symbol in the **Inputs** section of the Property panel on the right

Name	Value
Local Variable Name	carType

Name	Value
Variable Assignment Value	carType



5. Add two new **Output Variables** by clicking on the **+** symbol in the **Outputs** section of the Property panel on the right

Name	Value
Process Variable Name	riskCode
Variable Assignment Value	risk.code

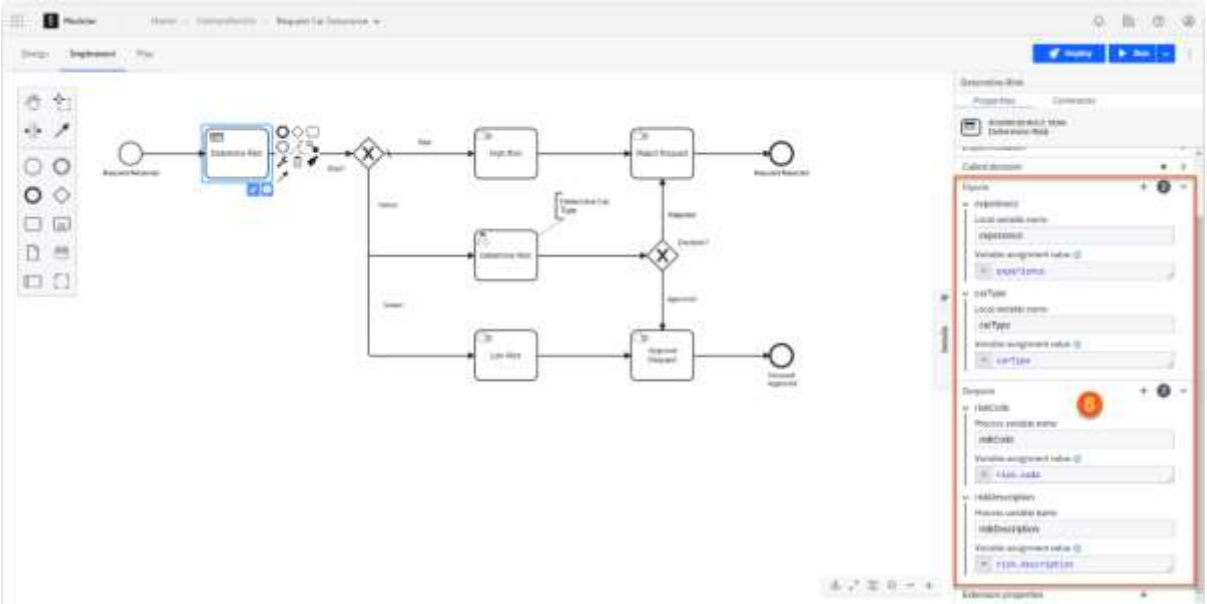
Name	Value
Process Variable Name	riskDescription
Variable Assignment Value	risk.description

Output Variables

The previous version of the **Decision Table** returned a single value that was directly mapped to the *risk* result (scalar variable).

The new version of the **Decision Table** is returning a *risk* object which contains two attributes; *code* and *description*. These are now being mapped to process variables.

5. The outputs from the **Business Rule Task** should now look as follows:



Modify the Gateway Conditions

We have changed the name of the *risk* variable to provide more clarity in our process; we now have *riskCode* and *riskDescription* which clearly capture the decision.

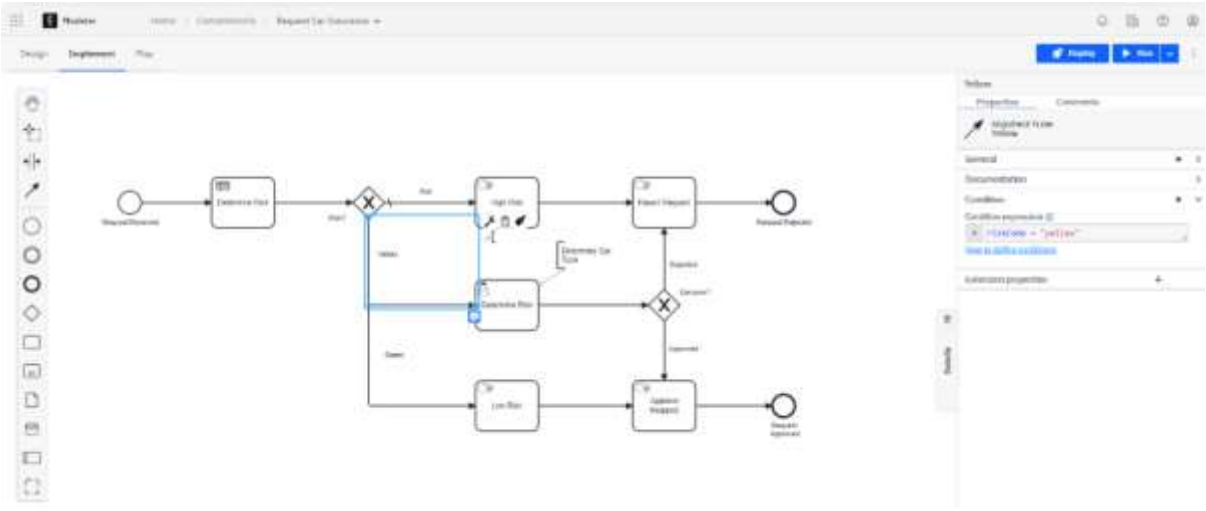
We now need to update the conditional sequence flows to reflect the new variables.

1. Select the sequence flow leading to *Low Risk* (labelled "Green") and modify the **Condition Expression** as below:

Name	**Value**
Condition Expression	riskCode = "green"

2. Select the sequence flow leading to *Determine Risk* (labelled "Yellow") and modify the **Condition Expression** as below:

Name	**Value**
Condition Expression	riskCode = "yellow"



Deploy Process Definition

Finally, let's re-deploy our updated process. A new version of the **Process Definition** will be created as we have already deployed the previous version.

1. Click on the **Deploy Diagram** button in the top right corner of the **Modeler**
2. Ensure that the *Camundanzia* **Cluster** is selected from the dialog box that appears.
3. Note that you are also reminded to deploy your **Decision Requirements Diagram** as it is now linked to the **Process Definition**
4. Click the **Deploy** button to deploy the **Process Definition** to the **Cluster**
5. A **Diagram Deployed** message will be displayed at the bottom of the **Modeler** to show that this was successful.

Deployed Process Definition

Congratulations! You have now deployed the updated *Request Car Insurance* **Process Definition** into the **Zeebe Engine**.

Cleanup Environment

Overview

We have now met all of the business requirements that have been defined in our **Scenario**. We are now in a position where we can perform an end-to-end test of our process to ensure that everything is working as expected.

However, as we have been testing our process definition as we have been developing it, it is likely that we have several incomplete process instances still running. Having incomplete (and possibly incorrect) process instances running in our environment may make it more difficult to confirm that everything is working as expected in an end-to-end test.

We will now **Cancel** and **Delete** any remaining instances so that we have a completely clean environment in which we can conduct our acceptance tests.

Cancel & Delete Process Instances

Launch Operate

Launch **Operate** by following the steps below:

1. From the **Console**, select **Clusters**
2. Select *Camundanzia* from the **Clusters** page
3. Click the **Launch** link next to **Operate** in the **Cluster Details** page

4. After a few seconds, the **Operate** Dashboard will be displayed

Cancel Process Instances

We can now **Cancel** the process instances in **Operate** by following the steps below:

1. Click on *Request Car Insurance* from the **Instances by Process** panel on the left-hand side of the screen
2. On the bottom left of the screen is a series of filters that will allow you to select *Running* or *Finished* process instances
3. Ensure that the *Running* filters are selected and the *Finished* filters are not selected
4. Then, for each **Process Instance** that is shown in the **Instances** panel, click the **Cancel Instance** button to the right-hand side of the process

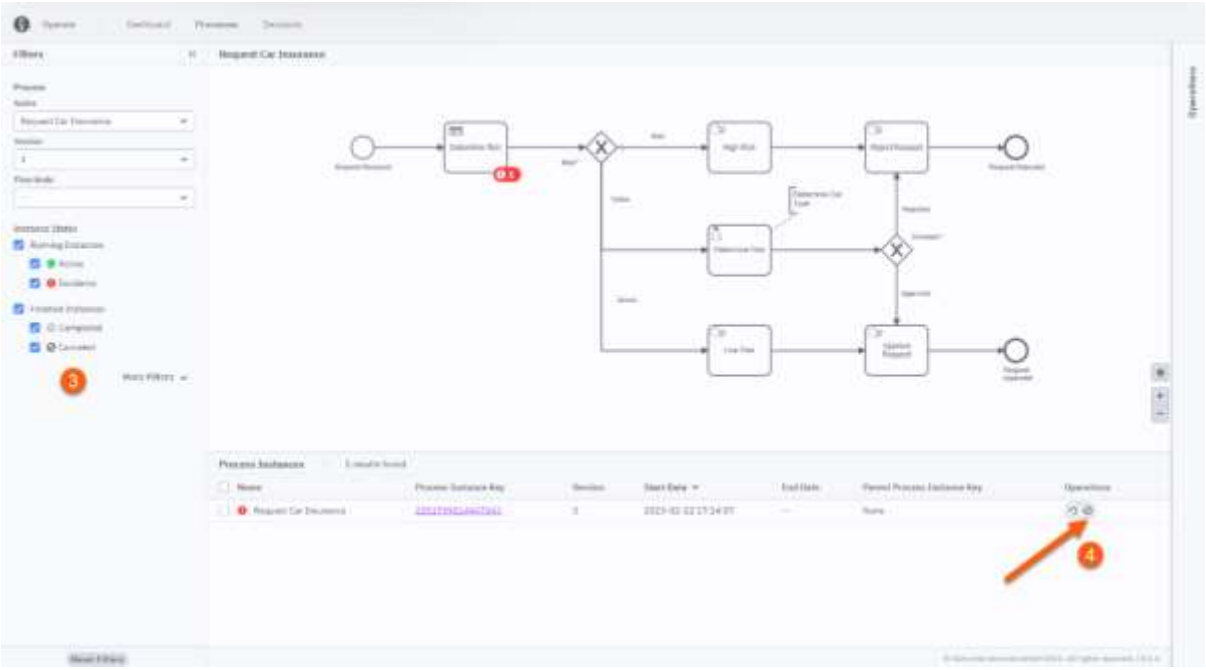
   No Running Processes

   This section can be skipped if there are no running processes shown in the **Instances** panel.

5. The **Process Instance** will then be cancelled and will disappear from the **Instances** panel

   What happened to the Process Instance?

   When you **Cancel** a process in Operate it changes to a stopped state. As the process is no longer running it is not picked up by *Running* filter.



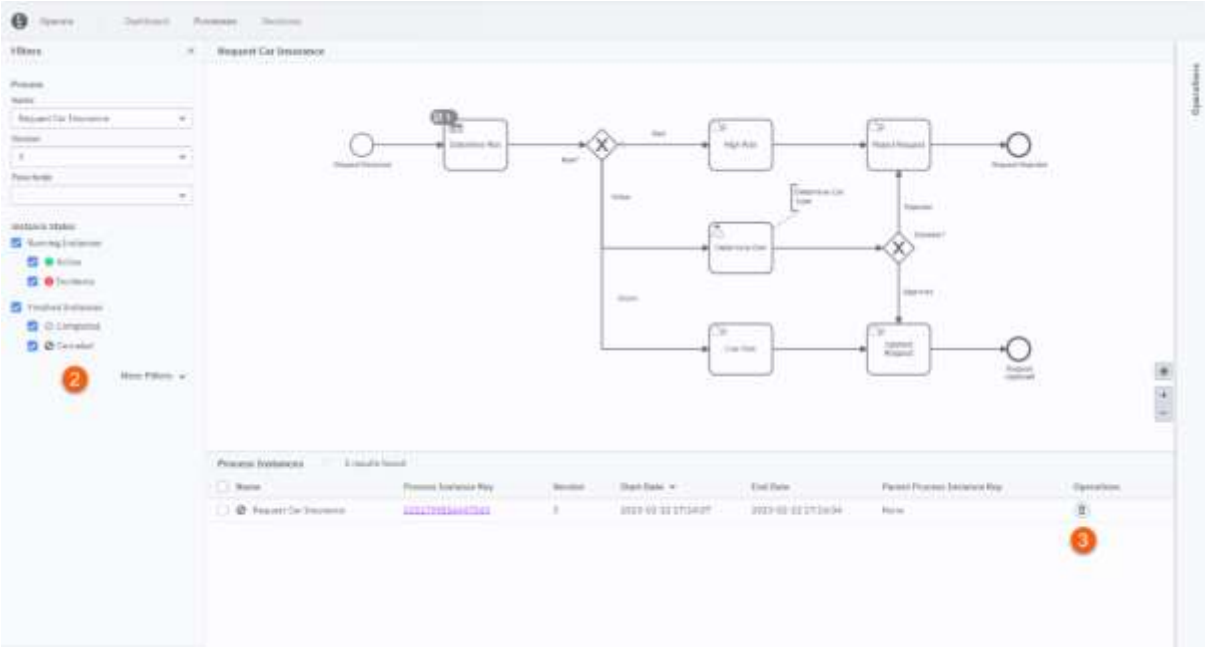
Delete Process Instances

We can now **Delete** the finished process instances in **Operate** by following the steps below:

1. On the bottom left of the screen is a series of filters that will allow you to select *Running* or *Finished* process instances
2. Ensure that the *Finished* filters are selected and the *Running* filters are not selected
3. Then, for each **Process Instance** that is shown in the **Instances** panel, click the **Delete Instance** button to the right-hand side of the process
4. The **Process Instance** will then be deleted and will disappear from the **Instances** panel

What happened to the Process Instance?

When you **Delete** a process in Operate it is completely removed from the **Zeebe Engine**.



Cleaned the Environment

Congratulations! You have now removed all instances of *Request Car Insurance* from the environment.

Test Process

We are now ready to test the *Request Car Insurance* process.

Review Acceptance Criteria

Let's start by reviewing the acceptance criteria that was defined earlier; we need to ensure that all of the requirements below can be met by our process.

1. Requests should be rejected for a *Family* or *Luxury* car if the **Driver** has less than 2 years of experience
2. Requests should be accepted for a *Small* car if the **Driver** has more than 2 years of experience
3. Requests should be accepted if the **Driver** has more than 4 years of experience
4. All other requests will be determined by an **Insurance Specialist**
5. The **Insurance Manager** can review each *Policy Request* after completion

The **Process Definition** and **Decision Table** that you have configured ensures that the first four acceptance criteria are met; the features and functionality available in **Operate** meets the final criterion.

Test Application

Test Scenarios

The following scenarios will be required to fully confirm that the acceptance criteria above has been met.

Test Scenario	Experience	Car Type	Expected Result
1 - High Risk Car (Family & 1 Years)	1.0	"family"	"red"
2 - High Risk Car (Luxury & 1 Years)	1.0	"luxury"	"red"
3 - High Risk Car Edge Case (Luxury & 2 Years)	2.0	"luxury"	"red"
4 - Low Risk Car (Small & 3 Years)	3.0	"small"	"green"
5 - Low Risk Car Edge Case (Small & 2 Years)	2.0	"small"	"yellow"
6 - Low Risk Driver (Luxury & 5 Years)	5.0	"luxury"	"green"

Test Scenario	Experience	Car Type	Expected Result
7 - Low Risk Driver Edge Case (Luxury & 4 Years)	4.0	"luxury"	"yellow"

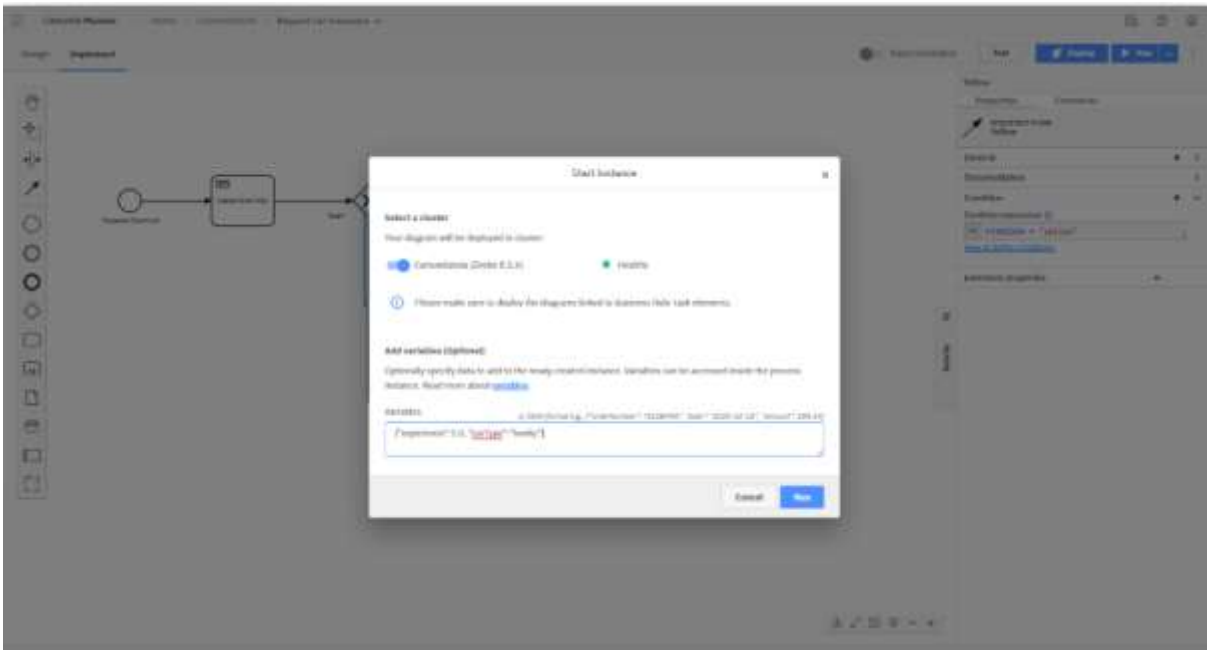
Launch Process Instances

Start seven instances of the *Request Car Insurance* process by following the steps below:

1. Click on the **Run** button in the top right corner of the **Modeler**
2. Ensure that the *Camundanzia* **Cluster** is selected from the dialog box that appears.
3. We need to start the process with a value for the driver's *experience* and the *carType*; enter the following JSON string for each instance into the **Add Variables** section

Test Scenario	JSON
1 - High Risk Car (Family & 1 Years)	{"experience":1.0, "carType":"family"}
2 - High Risk Car (Luxury & 1 Years)	{"experience":1.0, "carType":"luxury"}
3 - High Risk Car Edge Case (Luxury & 2 Years)	{"experience":2.0, "carType":"luxury"}
4 - Low Risk Car (Small & 3 Years)	{"experience":3.0, "carType":"small"}
5 - Low Risk Car Edge Case (Small & 2 Years)	{"experience":2.0, "carType":"small"}
6 - Low Risk Driver (Luxury & 5 Years)	{"experience":5.0, "carType":"luxury"}
7 - Low Risk Driver Edge Case (Luxury & 4 Years)	{"experience":4.0, "carType":"luxury"}

4. Click the **Run** button to start a new instance of *Request Car Insurance* using the *Camundanzia* **Cluster**

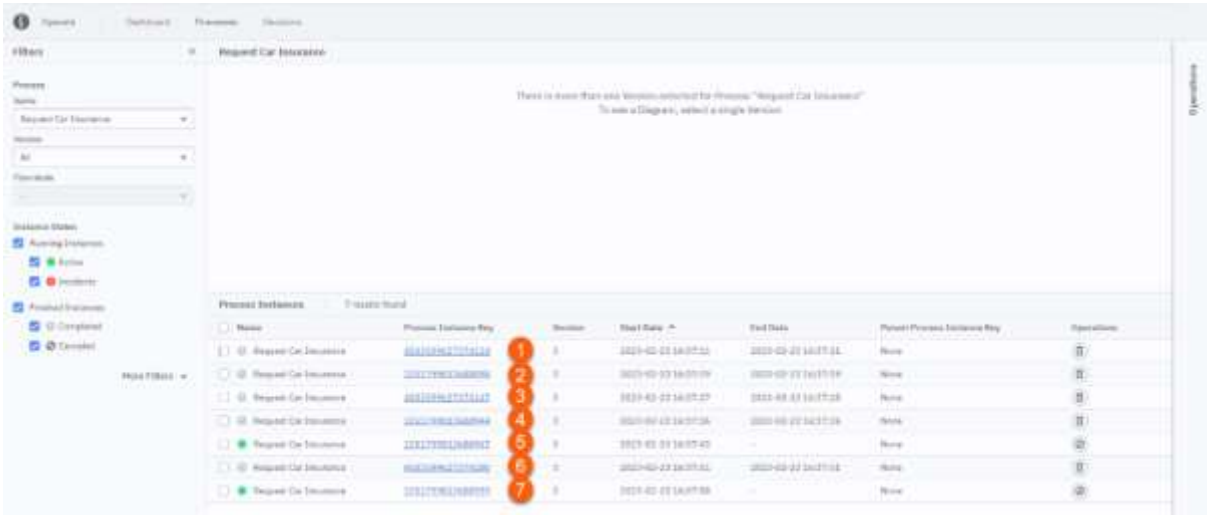


5. An **Instance Started!** message will be displayed at the bottom of the **Modeler** to show that this was successful.

Review Process Instances

We can now review the process instances in **Operate** by following the steps below:

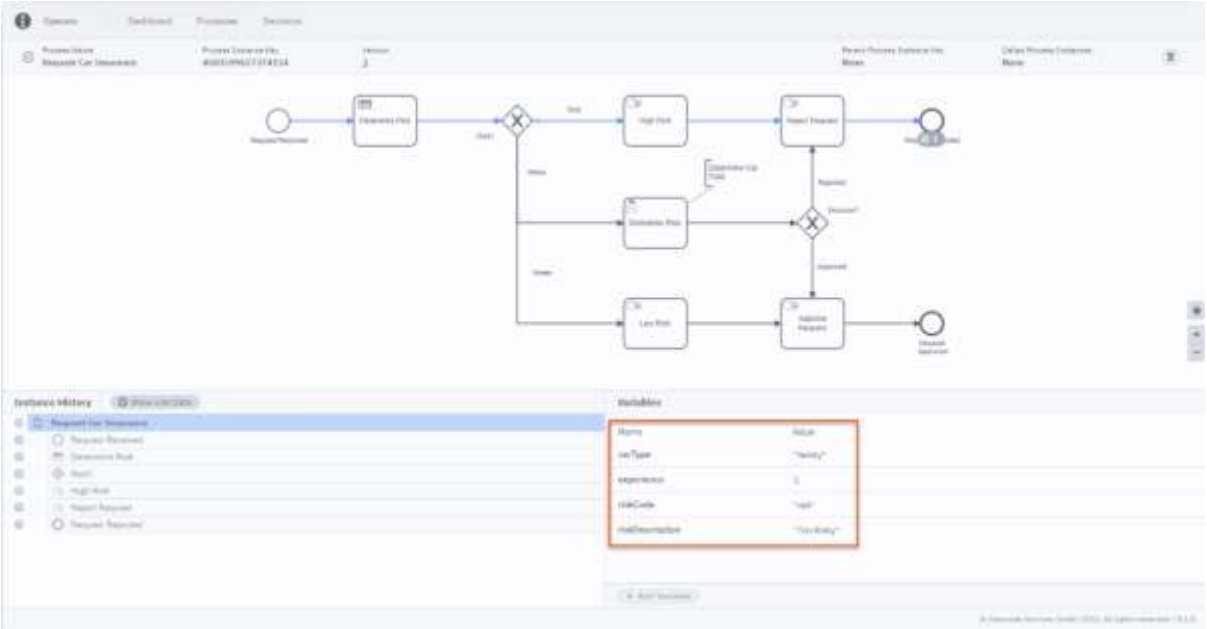
1. Click on *Request Car Insurance* from the **Process Instances by Name** panel on the left-hand side of the screen
2. **Operate** should now be showing 7 processes in the **Process Instances** panel. If everything has been configured correctly, 5 of these will be complete and 2 will still be running.
3. Sort the process instances by ascending **Start Date**; you should see a series of results similar to the below:



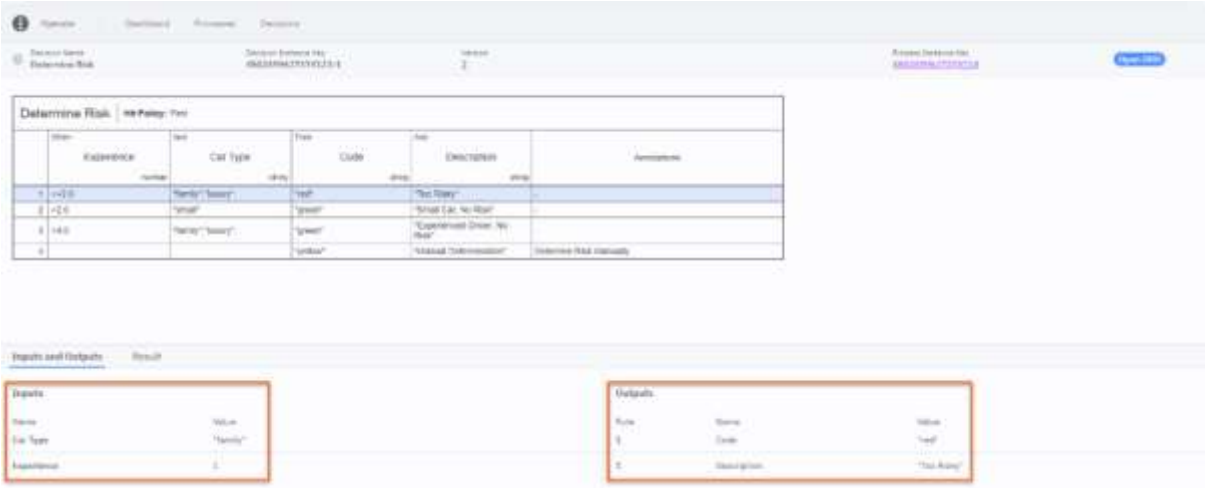
No Process Instances

If you are unable to see any process instances, check whether the filters on **Instance States** have been configured correctly

4. Click on the first **Process Instance Key** at the top of the list to open up the **Process Instance View**



5. We can now see that the process completed with the *Application Rejected* event, that the *code* was set to *red*, the *description* was set to *Too Risky* and that the process instance followed the expected path
6. Click on *Determine Risk* in the process diagram and then the **Called Decision Instance** link to open the **Decision Instance View**



7. We can now see the **Inputs**, **Outputs** and **Results** of the **Decision Table**. As expected, Rule 1 was triggered and the value returned was *red*.
8. Repeat steps 4 to 7 for each process instance to confirm that the results obtained are as per the **Test Scenarios** above

Deployed a Process Definition

Congratulations! You have now completed the testing of your process and have confirmed that it meets the acceptance criteria defined at the start of this course.

Example Solution

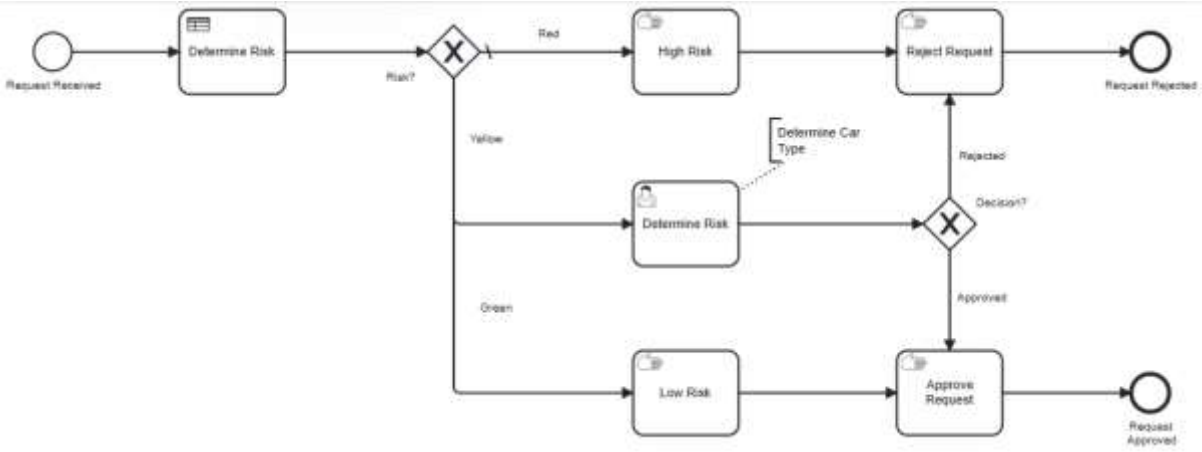
Solution

If you encountered any problems during this course or were unable to complete the process and decision configurations, you can compare your solution with our example solution below.

Process Definition

Our version of the **Process Definition** including the **Form Definition** can be found in the GitHub Repository below:

- Solution - Process Definition



Decision Requirements Diagram

Our version of the **Decision Requirements Diagram** can be found in the GitHub Repository below:

- Solution - Decision Requirements Diagram

Determine Risk | Hit Policy: First

	When	And	Then	And	Annotations
	Experience (number)	Car Type (string)	Code (string)	Description (string)	
1	<=2.0	"family","luxury"	"red"	"Too Risky"	
2	>2.0	"small"	"green"	"Small Car, No Risk"	
3	>4.0	"family","luxury"	"green"	"Experienced Driver, No Risk"	
4	-	-	"yellow"	"Manual Determination"	Determine Risk Manually
+					